TP 15 : Algorithme de Karger

Ce TP est à réaliser en C.

Préliminaires

On considère le problème `Coupe minimale` suivant :

Instance :	Un graphe connexe G non orienté
Solution :	Une partition des sommets en deux ensembles A et B
Fonction à minimiser :	Le nombre d'arêtes entre A et B.

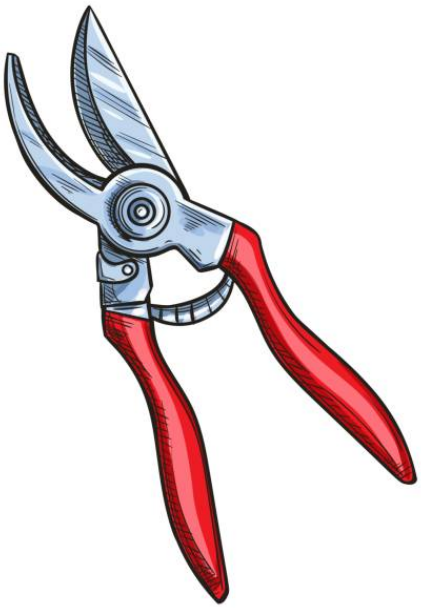

FIGURE 1 – Un joli sécateur pour découper votre graphe.

L'objectif de ce TP est d'implémenter un algorithme probabiliste pour répondre à ce problème.

Algorithme 1 Algo(G)

tant que Le graphe G contient strictement plus de 2 sommets **faire**
  Choisir une arête (s,t) du graphe uniformément.
  La contracter.
  Réaliser l'union de l'ensemble des sommets de s et de t.
fin tant que
return Les deux ensembles de sommets.

Détaillons un peu l'opération de contraction d'une arête :
Contracter l'arête (s,t) dans le graphe G revient à créer le graphe G' vérifiant :

- $S(G') = S(G) \setminus \{s,t\} \cup \{(s,t)\}$
- $A(G') = \{(s,t) \in S(G')^2 | (S,t) \in A(G)\} \cup \{((s,t),r) | (s,r) \text{ ou } (t,r) \in A(G)\}$

Moins formellement, on considère s et t comme un unique sommet dans G' et l'on conserve les arêtes résultant de cette considération. Les voisins du sommet contracté sont l'union des voisins des sommets de départ. Attention, il peut ainsi y avoir plusieurs arêtes entre s et t, on veille à toutes les conserver en pondérant l'arête entre s et t avec le nombre d'arêtes présentes en réalité.

On rappelle qu'en C, la fonction `int rand(void)` de la bibliothèque `stdlib.h` renvoie un entier compris entre 0 et `INT_MAX`. On initialise la graine aléatoire à l'aide de la fonction `void srand(int seed)` qu'on appellera souvent de la façon suivante : `srand(time(NULL))`

Des opérations unitaires

On considère par la suite un graphe sous forme de matrice d'adjacence. On utilisera donc le type `int **` pour représenter un graphe.

Question 1 : Écrire une fonction `int** graphe_vide(int n)` qui, étant donné un entier n, renvoie la représentation du graphe vide à n sommets.

Question 2 : Écrire une fonction `void liberer_graphe(int** G, int n)` qui libère l'espace mémoire alloué pour un graphe à n sommets.

Question 3 : Écrire une fonction `void contracter(int** G, int n, int s, int t)` qui, étant donnés un graphe G, sa taille n et deux sommets s et t, modifie G pour qu'il corresponde au graphe obtenu par contraction. *On choisira l'un des deux sommets comme représentant de l'union.*

Question 4 : Écrire une fonction `int taille_A(int** G, int n)` qui, étant donné un graphe G de taille n, renvoie le nombre d'arêtes qu'il possède.

Question 5 : Écrire une fonction `int tirage(int** G, int n, int k)` qui, étant donnés un graphe G de taille n, renvoie la k-ième arête (s,t) du graphe sous la forme : $n*s+t$.

L'algorithme en lui-même

Question 6 : Écrire une fonction `void iteration(int** G, int n)` qui, étant donnés un graphe et son nombre de sommets, applique une itération de la boucle `TantQue`.

Question 7 : Écrire une fonction `int karger(int** G, int n)` qui, étant donné un graphe G de taille n, renvoie la taille de la coupe obtenue par l'algorithme de Karger.

L'implémentation actuelle ne permet pas de retrouver la trace des ensembles A et B de sommets. On se propose dans la prochaine partie d'y remédier :

Reconstruire les ensembles.

On propose la structure de donnée suivante nous permettant de reconstruire la partition :

```
typedef struct Chainons
{
    int valeur;
    struct Chainons* suivant;
    struct Chainons* precedent;
}Chainon_circulaire;

typedef struct partition_t
{
    int n;
    Chainon_circulaire* tab;
}partition;
```

Le type `Chainons_circulaire` implémente un ensemble comme une liste doublement chaînée circulaire. Le type `partition` utilise un tableau `tab` de taille n où chaque case contient un chaînon. Si la partition contient le sous-ensemble $\{2,5\}$, la case 2 possédera comme successeur et prédécesseur un pointeur vers la case 5 et inversement.

Question 8 : Écrire une fonction `partition initialiser(int n)` qui crée une instance de type `partition` correspondant à la situation avant toute contraction.

Question 9 : Écrire une fonction `void contracter_circulaire(int** G, int n, partition P, int s, int t)` qui réalise la contraction de l'arête (s,t) en répercutant celle-ci sur la partition P.

Question 10 : Écrire une fonction `partition coupe(int** G, int n)` qui réalise l'algorithme de Karger sur un graphe G de taille n et renvoie la partition ainsi obtenue.

Question 11 : Écrire une fonction `bool* karger_circulaire(int** G, int n)` qui renvoie la partition sous forme d'un tableau de booléens. Deux éléments seront dans le même ensemble de la partition si et seulement si les cases correspondantes ont même valeur.

En pratique, l'algorithme précédent est répété n^2 fois et la coupe de plus petite taille obtenue est celle conservée.

Question 12 : Écrire une fonction `bool* karger_repetition(int** G, int n)` qui, étant donné un graphe G de taille n, renvoie la meilleure partition obtenue.